



DENA-CORE services for [Admins]

Ficha del Documento

Author	DENA-NEU support office
Document Description	DENA-CORE services for [Admins]

Version control

Versión	Date	Author	Description

TABLE OF CONTENTS

1	CORE SERVICES FOR [ADMINS].....	4
2	SEND INFO ABOUT CHANGES IN DATA (SRMD)	5
2.1	HIGH LEVEL VIEW	5
2.2	IMPLEMENTATION EXAMPLE	8
2.3	DATA INTERCHANGED BETWEEN THE [ADMIN] AND DENA-CORE	11
2.3.1	SRMD sent from [admin] to DENA-CORE	11
2.3.2	SRMD received from DENA-CORE as response	12
3	EXPOSE A [SERVICE] FOR [DATA RETRIEVAL]	13
3.1	HIGH LEVEL VIEW	13
3.2	IMPLEMENTATION EXAMPLE	2
3.3	DATA RETURNED BY THE [ADMIN] [DATA PROVIDER]	5
4	PERSON SYNC.....	6
4.1	DENA PUSH.....	7
4.2	ADMIN PULL.....	8
4.2.1	Online Admin pull	8
4.2.2	Offline	11
5	INTEGRATION REFERENCE ARCHITECTURES	15
5.1	PERSON SYNC	16
5.2	SRMD SYNC METADATA SENDING	16
5.3	DATA RETRIEVE.....	16

1 CORE SERVICES FOR [ADMINS]

This document describes the **services offered by DENA-CORE for [Admins]**

The CORE-Services are:

1. **[SRMD Sync]**: a mean for the [admins] to send [changes] in [data] to DENA-CORE
2. **[Person Sync]**: a mean for [admins] to have the [person data] of [persons] with a [DENA account]

Besides the CORE-Service offered by DENA, this document describes the **[data retrieve]** services that the [admins] have to expose to provide access to the full [data] of a certain [data type] for a [person]

It's important to understand the DENA architecture basis and DENA configuration before diving in this document

In particular:

- The DENA-Concepts
- The SYNC and RETRIEVE mechanism

2 SEND INFO ABOUT CHANGES IN DATA (SRMD)

2.1 High level view

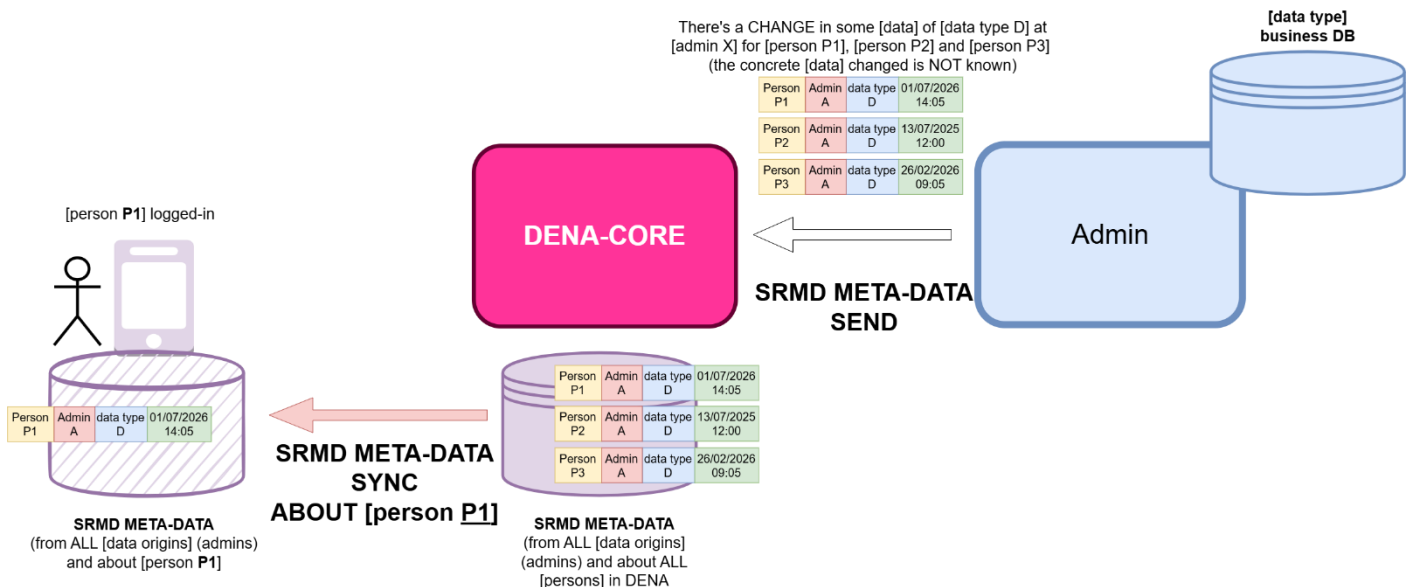
Every [admin] offers multiple [data types] in the DENA ecosystem.

As described in the [architecture] document:

- The [client app] can retrieve the full [data] from their [data origins] in the [admins]
- ... but to **avoid non-necessary [data retrievals]** and to make [client app] **responsive and usable**:
 - [client app] **caches** the retrieved [data] in a [local cache] and uses it **while it remains valid** (*it's not changed in the [data origin]*)
 - [client app] only issues a [data retrieval] for **refreshing its [local cache]** if it knows that the [data origin] (admin) has **[changes]** the [client app] has not retrieved
- ... so a **[change detection] mechanism** is needed so that the [client app] can know about [changes]

The [change detection] mechanism is the **[sync and retrieve meta-data] synchronization**

A high-level view (*read it from right to left*):



The [admin] **periodically** (as often as possible):

1. Queries the [business DB] (*the DB where the [data] resides*) **looking for items of [data] (rows) that have changed** since the last cycle
2. For every item of [data] (row) that has changed (*only for changed items*) create a data structure (SRMD) like:

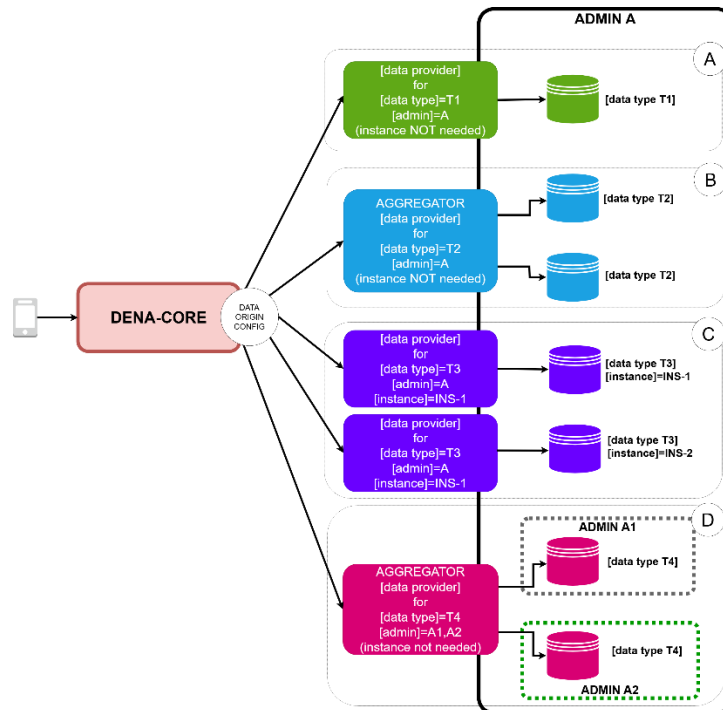
```
{person} | {data type} | {admin} | {last update instant}
```

3. The [admin] sends ALL the SRMD to DENA-CORE using a REST-service exposed by DENA-CORE

... **that's all for the [admin]**

Beware that the [admin] can send DENA-CORE SRMD for multiple [data types] and multiple [admins] in the same operation / message / package

This enables different strategies that can be used by the [admins] to send SRMD to DENA-CORE:



- A [data provider] for a **single [data type] in an [admin]**
At the [admin] side, a **dedicated** [data provider] offers the [retrieval service] for the [data type] in the [admin]
- A [data provider] that **aggregates** [data] of a certain [data type] from **multiple origins at different [instances] of a system** (for example, multiple [procedure record management systems] -gestores de expedientes- to create an aggregated [citizen view])
- Multiple [data providers] for multiple origins at different [instances] of a system** (for example, multiple [procedure record management systems] -gestores de expedientes- but with an split view or them)
- A [data provider] that aggregates [data] of a certain [data type] from multiple origins in different [admins] (for example, supra-territorial orgs like [diputaciones forales] that provides [data] in the SAME [data provider] for multiple [ayuntamientos])

Also is important to note that usually SRMD is like:

{person} | {admin} | {data type} | {last update instant}

However, in **strategy [c]**:

- An [admin] has **multiple [data origins]** for a given [data type] (multiple origin systems for the same [data type])
- The [admin] exposes a **dedicated [data provider] for each [data origin]**

In this architecture, the SRMD record have to provide an **additional metadata** field to provide **routing information** to DENA to get to the [data provider]. That metadata field is the **[data origin instance]**

[person] | [admin] | [data type] | [data origin instance] | [last update instant]

For its part, the [client device] (DENA-APP) performs the following steps:

1. DENA-APP sends ALL locally-stored SRMD for the [person] identified in DENA-APP (*and received in previous SRMD syncs*)
2. DENA-CORE
 - a) Loads the SRMD received from [admins] for that [person]
 - b) Compares it with the SRMD received from DENA-APP
 - c) Applies the following logic to guess if DENA-APP should issue a RETRIEVE operation for each [data type]

SRMD is like:

{person} | {data type} | {admin} | {last update instant}

which mean that “some” [data] for [person] of [data type] has been modified (*or is new*) in the given [admin]

(*the concrete [data] modified is NOT known; **only that a [change] in the [data] has occurred is known***)

If DENA-APP sends something like:

{person} | {data type A} | {admin X} | {T1}

{person} | {data type B} | {admin X} | {T2}

{person} | {data type C} | {admin Y} | {T3}

And the SRMD stored in DENA-CORE received from the [admins] is like:

{person} | {data type A} | {admin X} | {T1}

{person} | {data type B} | {admin X} | {T2+1min}

{person} | {data type C} | {admin Y} | {T3+5min}

Then DENA-CORE can conclude that:

- [data type A] in [admin X] has NOT been modified: T1 received from DENA-APP = T1 at DENA-CORE
- [data type B] in [admin X] has been modified: T2 received from DENA-APP < T2+1min at DENA-CORE
- [data type C] in [admin Y] has been modified: T3 received from DENA-APP < T3+5min at DENA-CORE

... so DENA-CORE can instruct DENA-APP that it has to **refresh (*retrieve fresh [data] from the [admin]*)** for:

- [data type B]
- [data type C]

[data type A] does NOT have to be refreshed since there's no indication of a [change] in the [data origin] (*admin*)

2.2 Implementation example

The first step for an [admin] to SEND meta-data to DENA-CORE is to **query for [changes] in the [data types]** ... it's up to the [admin] to get these [changes] and usually is just a simple SQL query for every [data type] since **the only info needed by DENA-CORE is**

- **whether there has been any [change] in any [person]-related [data] (rows / records) of a [data type]**
- **when was the LAST [change]**

(new or deleted row / records also accounts as [change])

The concrete [person]-related [data] (row / record) that has [changed] is NOT important; JUST the fact that a [change] has occurred in the [data origin] and when

As an example, take a [data type] that is backed by a [business DB] with the following structure:

PERSON_ID	PERSON_DATA	CREATED_AT	LAST_UPDATED_AT
48291038Z	What ever data	2026-08-17T03:22:10.150Z	2026-08-17T09:14:22.481Z
10593847H	What ever data	2026-08-17T14:05:49.882Z	2026-08-17T18:41:03.109Z
72940182C	What ever data	2020-09-15T08:30:12.400Z	2021-03-04T12:05:44.821Z
39502814Y	What ever data	2023-01-10T19:42:05.118Z	2024-11-22T04:19:30.954Z
84120395D	What ever data	2015-11-03T11:14:55.620Z	2018-07-09T21:58:12.337Z
...	...	...	...

The SQL that returns rows that have changed AFTER **2026-08-17T00:00:00.000Z** is quite simple:

```
SELECT PERSON_ID, MAX(COALESCE(LAST_UPDATED_AT,CREATED_AT)) AS LAST_CHANGE_AT
FROM DB_TABLE
WHERE COALESCE(LAST_UPDATED_AT,CREATED_AT) >= TIMESTAMP '2026-08-17T00:00:00.000Z '
GROUP BY PERSON_ID;
```

Note that COALESCE(LAST_UPDATED_AT,CREATED_AT) uses LAST_UPDATED_AT if present, falling back to CREATED_AT if LAST_UPDATED_AT is NULL

The SQL query should return something like:

```
PERSON_ID | LAST_CHANGE_AT
-----+-----
48291038Z | 2026-08-18 14:30:00.000
10593847H | 2026-08-17 11:20:00.000
72940182C | 2026-08-19 08:10:00.000
```

A simple implementation might be like the following:

```
public record DBPersonLastChange(String personId,
                                Instant lastChangedAt) {
    // db row carrier
}

private Collection<DBPersonLastChange> retrieveDBPersonsChangesFrom(final Instant from) {
    String sql = """
        SELECT PERSON_ID,
               MAX(COALESCE(LAST_UPDATED_AT,CREATED_AT)) AS LAST_CHANGE_AT
        FROM DB_TABLE
        WHERE COALESCE(LAST_UPDATED_AT,CREATED_AT) >= ?
        GROUP BY PERSON_ID
        """

    List<DBPersonLastChange> dbChanges = new ArrayList<>();
    try (Connection conn = DriverManager.getConnection(dbUrl,
                                                       user,pass);
         PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setObject(1,from); // 1st param: the instant from
        try (ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                DN00PersonID personId = DN00PersonID.from(rs.getString("PERSON_ID"));
                Instant lastChangedAt = rs.getTimestamp("LAST_CHANGE_AT");

                // create the db record
                dbChanges.add(new DBPersonLastChange(personId,
                                                       lastChangedAt));
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return dbChanges;
}
```

Now the [DB data] is converted to DENA [model objects]

```
private List<DN00SyncMetaDataFromAdminToCOREItem> toSRMD(final Collection<DBPersonLastChange> dbChanges) {
    return dbChanges.stream()
        .map(dbChange -> DN00SyncMetaDataBuilder.syncMetaDataFromAdminToCOREBuilder()
            .atAdmin(DN00OrgAdminID.forId("MY_ADMIN"))
            .fromSingleDataOrigin()
            .aboutPerson(DN00PersonID.forId(dbChange.personId))
            .someDataWasUpdatedAt(dbChange.lastChangedAt)
            .ofType(DN00DataTypeID.forId("citizer_service_appointment"))
            .build())
        .toList();
}
```

The final step is just **SEND** the SRMD to DENA-CORE wrapped into a [interop message]

```
public static void main(String[] args) {
    // [1] - Get the [changes] from the [DB]
    Collection<DBPersonLastChange> dbChanges = retrieveDBPersonsChangesFrom(Instant.now());

    // [2] - Transform to SRMD
    List<DN00SyncMetaDataFromAdminToCOREItem> srmd = toSRMD(dbChanges);

    // [3] - Wrap into a request [interop message]
    // (since the [admin] is "initiating" the flow, the [message correlation oid] is generated here)
    DN00InteropContext ctx = DN00InteropContextBuilder.createMessageOfType(DN00InteropMessageType.ADMIN_SYNC_METADATA_REQ)
        .withCorrelationOID(DN00MessageCorrelationOID.supply())
        .fromAdministration(DN00OrgAdminID.forId("MY_ADMIN"))
        .build();

    DN00SyncMetaDataFromAdminRequestPayload payload = DN00SyncMetaDataFromAdminRequestPayload.from(items);
    DN00SyncMetaDataFromAdminRequestMessage reqMsg = new DN00SyncMetaDataFromAdminRequestMessage(ctx,
                                                                                               reqMsg);

    // [4] - Marshall to JSON
    JavaPackage denaPckg = DN00AppCodes.API_JAVA_PACKAGE; // or just JavaPackage.named("dena")
    Marshaller marshaller = MarshallerBuilder.findTypesToMarshallAtJavaPackages(denaPckg)
        .build();

    String jsonToBeSent = marshaller.forWriting()
        .toJSON(reqMsg);
}
```

```

// [5] - Send to DEND (http POST)
log.info("SRMD to be sent\n" +
        "{}",
        jsonToBeSent);

String responseJSON = postSRMDtoDENA(jsonToBeSent);

log.info("DENA processing response\n" +
        "{}",
        responseJSON);

// [6] - Marshall DENA response back to [model object]
// a) unmarshall the response [interop message]
DN00SyncMetaDataFromAdminResponseMessage respMsg = marshaller.forReading()
        .fromJSON(responseJSON,
        DN00SyncMetaDataFromAdminResponseMessage.class);

// b) unwrap the DENA-CORE processing info
DN00SyncMetaDataFromAdminCOREProcessingInfo resp = respMsg.getPayload()
        .getProcessingInfo();

log.info("{} SRMD items processed",
        resp.getProcessedOKItemsCount());
}

/**
 * Send the SRMD Json to DENA
 * (admin AUTHENTICATION in DENA is IGNORED for simplicity's shake)
 */
private String postSRMDtoDENA(final String jsonPayload) {
    String responseJSON = null;

    // [1] Create the HttpClient instance
    HttpClient client = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(10))
        .build();

    // [2] Build the POST HttpRequest
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create("https://api.dena.eus/srmd/"))
        .header("Content-Type", "application/json")
        .header("Accept", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
        .build();

    // [3] Send the request synchronously and receive the response
    try {
        HttpResponse<String> response = client.send(request,
            HttpResponse.BodyHandlers.ofString());
        responseJSON = response.body();

        // debug
        log.info("Returned Status Code: {}",
            response.statusCode());
        log.info("Response Body:\n" +
            "{}",
            responseJSON);
    } catch (IOException | InterruptedException e) {
        log.error("Error sending SRMD to DENA: {}",
            th.getMessage(), th);
        Thread.currentThread()
            .interrupt(); // Restore interrupted status
    }
    // [99] Return
    return responseJSON;
}

```

2.3 Data interchanged between the [admin] and DENA-CORE

2.3.1 SRMD sent from [admin] to DENA-CORE

```
[
{
  "admin" : {
    "id" : "EJGV"
  },
  "aboutPerson" : {
    "id" : "40404040H"
  },
  "someDataWasUpdatedAt" : "2026-08-17T15:14:07.0369127Z",
  "ofType" : {
    "id" : "ADMIN_NOTICE"
  },
  "fromDataOrigin": "DEFAULT"
},
{
  "admin" : {
    "oid" : "45165481-70E5-479A-AADB-2E681E3A011D"
  },
  "aboutPerson" : {
    "oid" : "501872FE-9A67-4AF8-BAAE-2E385119FF5F"
  },
  "someDataWasUpdatedAt" : "2026-08-17T15:14:07.0324014Z",
  "ofType" : {
    "id" : "ADMIN_NOTICE",
    "oid" : "0E4A5333-283B-46B3-8532-C6C4486C3C05"
  },
  "fromDataOrigin": "DEFAULT"
},
{
  "admin" : {
    "id" : "EJGV",
    "oid" : "45165481-70E5-479A-AADB-2E681E3A011D"
  },
  "aboutPerson" : {
    "id" : "40404040H",
    "oid" : "501872FE-9A67-4AF8-BAAE-2E385119FF5F"
  },
  "someDataWasUpdatedAt" : "2026-08-17T15:14:07.0369127Z",
  "ofType" : {
    "id" : "unknown-data-type-id",
    "oid" : "unknown-data-type-oid"
  },
  "fromDataOrigin": "DEFAULT"
}
]
```

As shown above, the [info] sent by DENA-APP to DENA-CORE in essence is:

{person}		{data type}+{data origin instance}		{admin}		{last update instant}
						---> someDataWasUpdatedAt
						-----> atAdmin
+		+				-----> ofType + data origin instance
						-----> aboutPerson

Beware that the [admin], [data type] or [person] can be identified using the OID (*DENA-assigned unique identifier*) or just the ID (*for example, the NIF in the [person] case*)

There's NO need for the [admin] to use DENA OIDs; just IDs can be used without a problem

2.3.2 SRMD received from DENA-CORE as response

When the [admin] send the SRMD in response receives the result of the SRMD processing in DENA-CORE:

```
{
  "transactionOid" : "20863FFE-0BEB-4079-BFA1-A1EAEEDB58FF",
  "receivedItemsCount" : 3,
  "processedOK" : [
    {
      "oid" : "4F4EF685-BA83-4BAB-80CD-6B01BBE7939C",
      "lastTransactionOid" : "20863FFE-0BEB-4079-BFA1-A1EAEEDB58FF",
      "admin" : {
        "id" : "EJGV",
        "oid" : "45165481-70E5-479A-AADB-2E681E3A011D"
      },
      "dataType" : {
        "id" : "ADMIN_NOTICE",
        "oid" : "0E4A5333-283B-46B3-8532-C6C4486C3C05"
      },
      "dataOriginInstanceId": "DEFAULT",
      "person" : {
        "id" : "40404040H",
        "oid" : "501872FE-9A67-4AF8-BAAE-2E385119FF5F"
      },
      "dataLastUpdatedAt" : "2026-08-17T15:14:07.0324014Z"
    },
    {
      "lastTransactionOid" : "20863FFE-0BEB-4079-BFA1-A1EAEEDB58FF",
      "oid" : "4F4EF685-BA83-4BAB-80CD-6B01BBE7939C",
      "admin" : {
        "id" : "EJGV",
        "oid" : "45165481-70E5-479A-AADB-2E681E3A011D"
      },
      "dataType" : {
        "id" : "ADMIN_NOTICE",
        "oid" : "0E4A5333-283B-46B3-8532-C6C4486C3C05"
      },
      "dataOriginInstanceId": "DEFAULT",
      "person" : {
        "id" : "40404040H",
        "oid" : "501872FE-9A67-4AF8-BAAE-2E385119FF5F"
      },
      "dataLastUpdatedAt" : "2026-08-17T15:14:07.0369127Z"
    }
  ],
  "processedNOK" : [ {
    "item" : {
      "admin" : {
        "id" : "EJGV",
        "oid" : "45165481-70E5-479A-AADB-2E681E3A011D"
      },
      "aboutPerson" : {
        "id" : "40404040H",
        "oid" : "501872FE-9A67-4AF8-BAAE-2E385119FF5F"
      },
      "ofType" : {
        "oid" : "unknown-data-type-oid"
      },
      "fromDataOrigin": "DEFAULT",
      "someDataWasUpdatedAt" : "2026-08-17T15:14:07.0369127Z"
    },
    "error" : "The [data type] with ref=unknown-data-type-oid;null could NOT be validated"
  } ]
}
```

As shown above, the [info] returned by DENA-CORE to the [app] in essence has

- A DENA-CORE assigned [transaction Oid]
- The **total** number of **received SRMD items**
- The received SRMD items that could be **successfully processed** with:
 - A DENA-CORE assigned OID for the SRMD item
 - The [transaction Oid] where the SRMD item was created
 - And the SRMD itself: {person} | {data type} | {admin} | {last update instant}

```
{
  "oid" : "4F4EF685-BA83-4BAB-80CD-6B01BBE7939C",
  "lastTransactionOid" : "20863FFE-0BEB-4079-BFA1-A1EAEEDB58FF",
  "admin" : {
    "id" : "EJGV",
    "oid" : "45165481-70E5-479A-AADB-2E681E3A011D"
  },
  "dataType" : {
    "id" : "ADMIN_NOTICE",
    "oid" : "0E4A5333-283B-46B3-8532-C6C4486C3C05"
  },
  "dataOriginInstanceId": "DEFAULT",
  "person" : {
    "id" : "40404040H",
    "oid" : "501872FE-9A67-4AF8-BAAE-2E385119FF5F"
  },
  "dataLastUpdatedAt" : "2026-08-17T15:14:07.0324014Z"
}
```

3 EXPOSE A [SERVICE] FOR [DATA RETRIEVAL]

3.1 High level view

The previous section explained the mechanism that allows DENA-APP to know what [data origins] have new/updated [data] that must be **retrieved from DENA-APP**

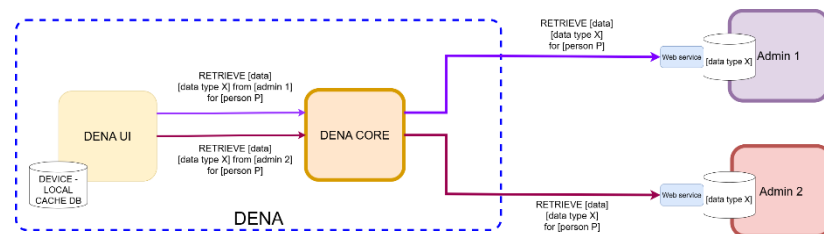
It's important to remember that:

Once the full [data] is retrieved from the [data origin] (*admin*), the [data] is **locally cached at DENA-APP**, so it **does not have to be retrieved again** if there's no indication of a [change] in the [data origin] (*admin*)

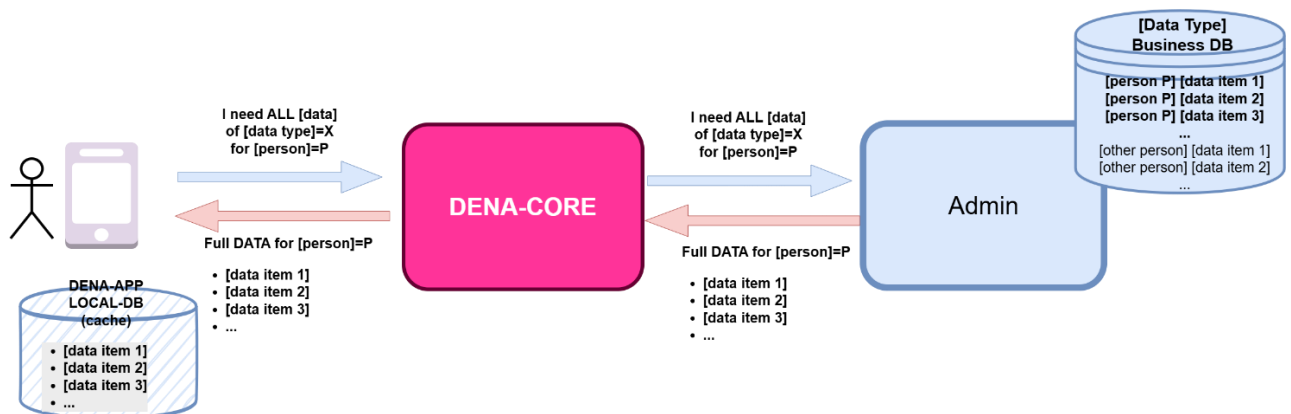
That locally-cached [data] is used by DENA-APP to “paint” [data] to the [user] as soon as possible

This section describes how that **retrieval** process is done.

The [data retrieval] is quite straightforward and just **uses DENA-CORE as a proxy (intermediator)** with the [data origin] (*admin*)



Another view of the previous:



As shown, **DENA-CORE acts as an intermediary / PROXY** for DENA-APP to retrieve the full [data] from the [admin]

This way, DENA-APP is **decoupled** from the [Admins] and do not need to know anything about them; all the [semantic] complexities are handled by DENA-CORE.

... so, DENA-APP always *talk* with DENA-CORE with **standard [semantics]**:

- DENA-CORE exposes a REST-service through HTTPS
- DENA-APP identifies itself at DENA-CORE using OAUTH
- The [data] interchanged for every [data type] adjust to a **standard format** (*each [data type] has it's own standard format that models the interchanged info*)

From the technical point of view:

- DENA-APP (DENA-UI) uses a **common [service] at [DENA-CORE]** where it “asks for” the [data]
- The [admin] must **expose a [data provider] service** that is called from [DENA-CORE] to retrieve the [data] for the DENA-APP

This proxy architecture **decouples DENA-APP from the multiple [service providers] at the [admins]: DENA-APP just *talks* with DENA-CORE.**

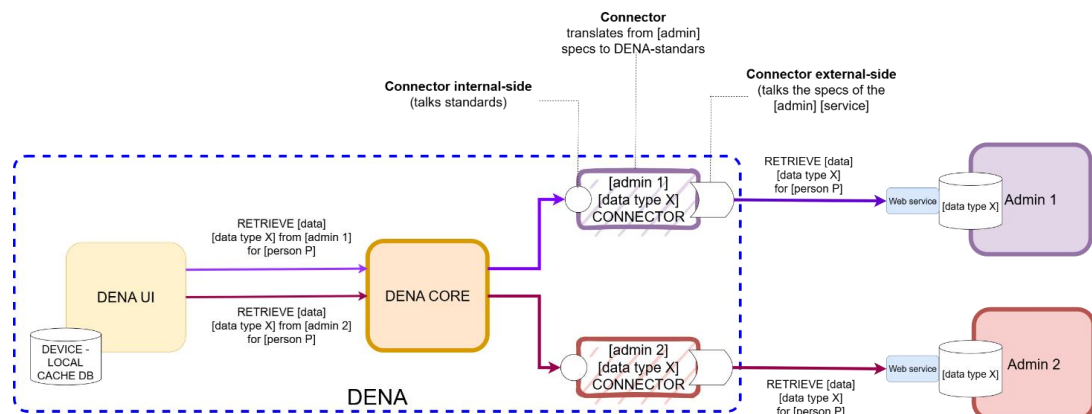
The implementation of the [data provider] at the [admin] that **provides [data] for retrieval** can choose between:

1. Build the [data provider] using the **DENA standard [semantics]**:
 - The [service] is exposed as an HTTP REST Service
 - DENA-CORE authenticates at the [admin] using OAUTH
 - Standard [data format] (*standard semantics*)
2. Build the [data provider] with **CUSTOM [semantics]** in one or many of the areas, for example:
 - [service] exposed with HTTP semantics different from a REST Service (*a SOAP-Web Service*)
 - DENA-CORE authenticates at the [admin] using an X-509
 - The [data format] (*semantics*) returned by the [data provider] does NOT align with the [standard]

Whatever approach the [admin] chooses (*standard impl or custom impl*) DENA-CORE needs to “something” that **translates** between:

- The [standard semantics] used internally in DENA-CORE and in DENA-APP
- The [semantics] (*custom or standard*) used at the [data provider] at the [admin]

DENA-CORE uses a [connector] architecture like the one depicted below:



The function of a [connector] is the **translation of the DENA [semantics] used by DENA-CORE to the [semantics] used by the concrete [data provider]** for [data] of a certain [data type] in an [admin]

A [connector] has two sides:

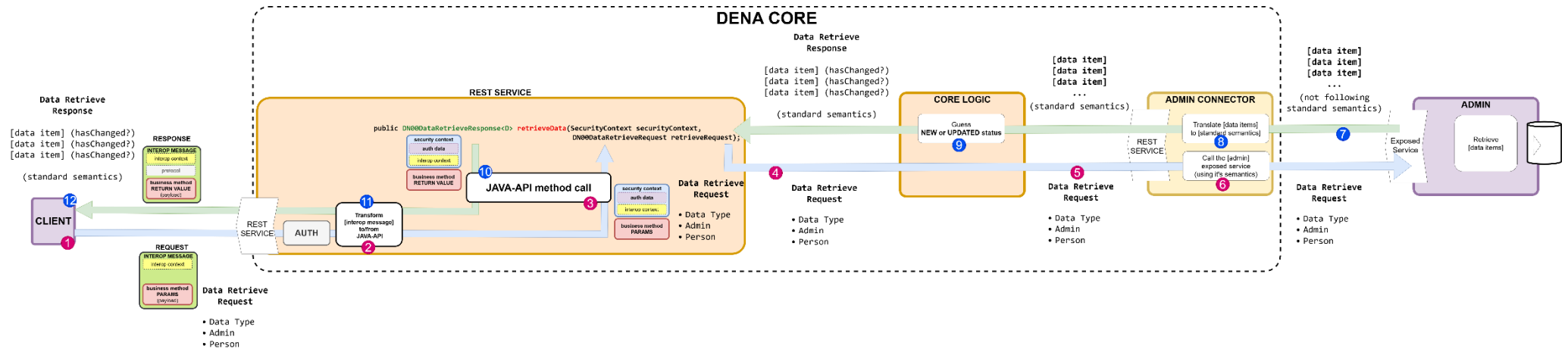
- The **[internal side]** that is used by the DENA-CORE using **DENA-standard [semantics]** (*transport, security and data format*)
- The **[external side]** that knows how to interact with the remote [data provider] service exposed by the [admin] in terms of:
 - **Transport:** *how to connect to the [remote service], for example the http(s)-specific details (url, urlpath, headers, etc)*
 - **Security:** *how DENA-CORE identifies itself at the [remote service], for example using an OAUTH service provided by the [admin] or a X509 certificate*
 - **Data Format:** *the interchanged [data] format*

The [connector] **translates** the [semantics] of both sides: the standard one used internally and the [admin]-specific one used externally

... anyway, an [admin] can expose a [data provider] that use **standard [semantics]** so the [connector] just acts as a transparent proxy from DENA-CORE to the remote [data provider]

[connectors] are **independent modules** that are deployed independently of DENA-CORE to make the whole system more **resilient**: if an [admin] [data provider] throughput starts to degrade and returns the response very slowly, connections from the [connector] can stuck and cause problems in the [connector] but never affect to anything else in DENA system.

The following figure resumes all the steps for a [data retrieve] request:



- [1] - The client calls a DENA-CORE REST-Service requesting the [data retrieval] sending an [interop message] that contains the [data type], [admin] and [person] whose data is required
- [2] - The REST-Service authenticates DENA-APP and **translates** the [interop message] to [model objects] used by the JAVA-API
- [3] - The JAVA-API **retrieve data** method is called
- [4] - The JAVA-API calls the CORE LOGIC
- [5] - The CORE LOGIC guess what [admin connector] has to be used and calls the REST-SERVICE exposed (*which talks standard semantics*)
- [6] - The [admin connector] **calls a [data provider] service exposed by the [admin]** that provides the [data tiems]
- [7] - The [data provider] exposed by the [admin] **retrieves de [data items]** from the [business DB] and returns them to the [admin connector] (*note that the returned data DO NOT HAVE to follow the [standard semantics]*)
- [8] - The [admin connector] -if necessary- **translates the returned [data items]** from an [admin]-custom [semantics] to the [standard semantics]
- [9] - The CORE LOGIC guess the “**has changed**” state of every [data item]
- [10] - The JAVA-API returns the [data items] (*in standard semantics*) with the “**has changed**” status
- [11] - The REST-Service **translates** the JAVA-API returned [data items] to a **response [interop message]**
- [12] - The response [interop message] arrives to the [client]

It's important to know **how DENA-CORE guess the [has changed] status of every [data item] returned by the [admin]**

DENA-APP needs to know if each [data item] is NEW or has been UPDATED since the last time the [person] used DENA-APP because with this information **DENA-APP can highlight the NEW or UPDATED [data items]**

The [admin] does NOT need to send if a [data item] is NEW or has been UPDATED because **DENA-CORE guess that state while [data items] passes through DENA-CORE during [data retrieval]** (*remember that **DENA-CORE does not store [data items]***)

The logic that DENA-CORE uses is:

1. Every [data item] returned by the [admin] has a **lastChangedAt** field with the last instant when that [data item] has changed at the [admin]
2. DENA-CORE loads the **last time the [person] issued a [data retrieval]** (*before this one*)
3. If the last time the [person] issued a [data retrieval] is **BEFORE** the [lastChangedAt] instant of the [data item], then the [data item] is NEW or has been UPDATED

The most important info that the [admin] must include in each [data item] is the **last time the [data item] has changed** (lastChangedAt)

NOTE that the {last time the [person] issued a [data retrieval]} is **loaded for the [person] and NOT for the [client installment]**; this way the behaviour is consistent across [client installments]

3.2 Implementation example

As an example, take system that contains **appointments** for an [admin]'s [data provider] (*ie the citizen care service*) that is backed by a [business DB] with the following simple *fictional* structure:

PERSON_ID	AT	DURATION_MIN	SUBJECT_ES	SUBJECT_EU	CREATED_AT	LAST_UPDATED_AT
48291038Z	2026-08-17T14:30:45.123	30	[es]-appmt A1	[eu]-appmt A1	2026-08-17T03:22:10.150Z	2026-08-17T09:14:22.481Z
48291038Z	2026-09-04T10:15:22.450	15	[es]-appmt A2	[eu]-appmt A2	2026-08-17T14:05:49.882Z	2026-08-17T18:41:03.109Z
72940182C	2026-09-28T16:08:49.812	10	[es]-appmt B1	[eu]-appmt B1	2020-09-15T08:30:12.400Z	2021-03-04T12:05:44.821Z
72940182C	2026-10-15T07:22:11.390	30	[es]-appmt B2	[eu]-appmt B2	2023-01-10T19:42:05.118Z	2024-11-22T04:19:30.954Z
72940182C	2026-11-12T20:54:03.678	60	[es]-appmt B3	[eu]-appmt B3	2015-11-03T11:14:55.620Z	2018-07-09T21:58:12.337Z
...	...	...	...	...	...	...

A [data provider] returning all [data] for a given [person] might listen for DENA-CORE requests at a REST end-point like (*this is just a sample url **non-standard** semantics*)

https://my-admin.eus/dena/retrieve/citizen_care_appointments/{personId}

so, if any **authorized client** requests [data] for [personId]= 72940182C the url should be:

https://my-admin.eus/dena/retrieve/citizen_care_appointments/72940182C

and the [data provider] just returns a JSON like:

```
{
  JSON with the DATA item B1
  JSON with the DATA item B2
  JSON with the DATA item B3
}
```

A simple implementation of the [admin] [data provider] could be:

```
public record DBData(String personId,
                    LocalDateTime at,int durationMins,
                    String subjectES,String subjectEU,
                    Instant lastChangedAt) {
    // db row carrier
}

private Collection<DBData> retrieveDBDataFor(final DN00PersonID personId) {
    String sql = """
        SELECT PERSON_ID,
               AT,DURATION_MIN,SUBJECT_ES,SUBJECT_EU,
               MAX(COALESCE(LAST_UPDATED_AT,CREATED_AT)) AS LAST_CHANGE_AT
        FROM DB_TABLE
        WHERE PERSON_ID = ?
        """
    List<DBData> dbData = new ArrayList<>();
    try (Connection conn = DriverManager.getConnection(dbUrl,
                                                       user,pass);
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1,personId.asString()); // 1st param
        try (ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                DN00PersonID personId = DN00PersonID.from(rs.getString("PERSON_ID"));

                // get the [appointment] data
                LocalDateTime at = rs.getObject("AT",LocalDateTime.class);
                int durationMins = rs.getInt("DURATION_MINS");
                String subjectES = rs.getString("SUBJECT_ES");
                String subjectEU = rs.getString("SUBJECT_EU");

                // instant where the [data item] has CHANGED is VERY IMPORTANT for DENA-APP (UI)
                Instant lastChangedAt = rs.getTimestamp("LAST_CHANGE_AT");

                // create the db record
                dbData.add(new DBData(personId,
                                     at,durationMins,
                                     subjectES,subjectEU,
                                     lastChangedAt));
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return dbData;
}
```

```
}
```

Now the [DB data] is converted to [model objects]; in this case DENA-provided standard [model objects] for [schedule] (**DN00ScheduleItem**) are used (*standard semantics*)

```
private List<DN00ScheduleItem> fromDBDataToReturnedDataItems(final Collection<DBData> dbData) {
    return dbData.stream()
        .map(dbDataItem -> {
            // -- Create the [schedule item] (standard semantic using DENA [model objects])
            DN00ScheduleItem schItem = new DN00ScheduleItem();

            // last change instant > this is very important for DENA-APP (UI)!!
            schItem.setLastChangedAt(dbDataItem.lastChangedAt);

            // the appointment id at the origin [admin]
            schItem.setId(DN00ScheduleItemID.forId("appmt123"));

            // the appointment instant and duration
            schItem.setAt(dbDataItem.at);
            schItem.setDurationMinutes(dbDataItem.durationMinutes);

            // the appointment subject
            schItem.setSubjectByLanguage(new LanguageTextsMapBacked()
                .add(Language.SPANISH, dbDataItem.subjectES)
                .add(Language.BASQUE, dbDataItem.subjectEU));

            // URL at the [admin] web site to get more info about the appointment
            Url urlEU = Url.from("https://my-admin.eus/citizen-service-appointments/appmt123?lang=eu");
            Url urlES = Url.from("https://my-admin.eus/citizen-service-appointments/appmt123?lang=es");

            schItem.setUrls(UrlCollection.create()
                .addItem(UrlCollectionItem.createFor(urlEU),
                    CollectionItemID.forId("main"),
                    Language.BASQUE),
                UrlCollectionItem.createFor(urlES),
                    CollectionItemID.forId("main"),
                    Language.SPANISH));

            return schItem;
        })
        .toList();
}
```

Beware the every returned [data item] **should** contain the URL at the [admin] web site where the [user] can find more details about the [appointment], change it or even cancel it.

This URL is set as an URL collection where every item has a [language] and an [id]

```
Url urlEU = Url.from("https://my-admin.eus/citizen-service-appointments/appmt123?lang=eu");
Url urlES = Url.from("https://my-admin.eus/citizen-service-appointments/appmt123?lang=es");

schItem.setUrls(UrlCollection.create()
    .addItem(UrlCollectionItem.createFor(urlEU),
        CollectionItemID.forId("main"),
        Language.BASQUE),
        UrlCollectionItem.createFor(urlES),
            CollectionItemID.forId("main"),
            Language.SPANISH));
```

The minimal implementation of the REST service for this url-semantics:

https://my-admin.eus/dena/retrieve/citizen_care_appointments/{personId}

```
@RestController
@RequestMapping("/dena/retrieve")
public class AdminCitizenServiceAppointmentsController {

    @GetMapping("/citizen_care_appointments/{personId}")
    public ResponseEntity<String> getScheduleItemsByPersonId(@PathVariable("personId") String personIdParam) {
        // [0] - get the params
        DN00PersonID personId = DN00PersonID.forId(personIdParam);

        // [1] - get data from DB
        List<DBData> dbData = retrieveDBDataFor(personId);

        // [2] - transform from DB to DENA semantics (format)
        List<DN00ScheduleItem>> retrievedItems = fromDBDataToReturnedDataItems(dbData);

        // [3] - convert to JSON
        String json = _marshaller.forWriting()
                                .toJSON(schItems);

        // [99] - return
        return ResponseEntity.ok(json);
    }
}
```

Note that to maintain the example simple, the REST function returns a String with the json
... but this is NOT the most “elegant” way; the REST function signature should be:

```
@RestController
@RequestMapping("/dena/retrieve")
public class AdminScheduleXController {

    @GetMapping("/schedule_x/{personId}")
    public ResponseEntity<List<DN00ScheduleItem>> getScheduleItemsByPersonId(@PathVariable("personId")
                                                                           String personIdParam) {
        ...
    }
}
```

... and let spring marshall / unmarshall the returned List<DN00DataRetrievedFromAdmin<DN00ScheduleItem>> using Jackson.

But DENA [model objects] uses a custom Jackson configuration and a **custom HttpMessageConverter** is needed to marshall / unmarshall DENA [model objects]

TODO document the custom HttpMessageConverter

3.3 Data returned by the [admin] [data provider]

The [admin]-exposed [data provider] that **provides [data] items of a given [data type] about a given [person]** in the example returns something like:

```
[
  {
    "id" : "appointment123",
    "lastChangedAt" : "2026-08-19T08:07:56.7423565Z",    // very important!!!
    "year" : "2026",
    "monthOfYear" : "8",
    "dayOfMonth" : "18",
    "hourOfDay" : "13",
    "minuteOfHour" : "2",
    "durationMinutes" : 30,
    "subject" : {
      "ENGLISH" : "an appointment",
      "SPANISH" : "una cita"
    },
    "urls" : [ {
      "id" : "main",
      "lang" : "BASQUE",
      "value" : "https://my-admin.eus/citizen-service-appointments/appointment123?lang=eu"
    },
    {
      "id" : "main",
      "lang" : "SPANISH",
      "value" : "https://my-admin.eus/citizen-service-appointments/appointment123?lang=es"
    } ]
  },
  {
    "id" : "appointment456",
    "lastChangedAt" : "2026-08-19T08:07:56.7423565Z",    // very important!!!
    "year" : "2026",
    "monthOfYear" : "10",
    "dayOfMonth" : "5",
    "hourOfDay" : "10",
    "minuteOfHour" : "25",
    "durationMinutes" : 30,
    "subject" : {
      "ENGLISH" : "another appointment",
      "SPANISH" : "otra cita"
    },
    "urls" : [ {
      "id" : "main",
      "lang" : "BASQUE",
      "value" : "https://my-admin.eus/citizen-service-appointments/appointment456?lang=eu"
    },
    {
      "id" : "main",
      "lang" : "SPANISH",
      "value" : "https://my-admin.eus/citizen-service-appointments/appointment456?lang=es"
    } ]
  },
  ...
]
```

As detailed before, this returned [data items] are processed at DENA-CORE:

1. **The [admin connector] translates the [semantics] returned by the [admin] to the [standard semantics].** *(in the example case, this is not necessary since the [admin] return the [standard semantics])*
2. **The CORE-LOGIC** guess if it's a [new or updated data item] so DENA-APP can highlight these items

4 PERSON SYNC

Every [person] that enrolls in DENA will have an **account in DENA** that will store:

- Person OID: *a DENA-generated unique identifier*
- Person ID: *the NIF*
- Full name: name, surname1, surname2
- Contact Data: address, phone, email...
- Preferred themes (*can be used in proactive services or to customize the UI experience*)_

This *simple* [data] collected for every [person] enrolled in DENA is **available to every [admin] participating in DENA**, so the system offers the [admin] means to sync [person] [data].

One of the purposes of **synchronizing [persons] with the [admins]** is to **make them aware of which [persons] are enrolled in DENA**. This ensures the [admin] sends only the **[SRMD meta-data]** for those [persons], and not for anyone who does not have a DENA **[account]**

DENA provides the following means to the [admins] to sync [person] data:

- **DENA-PUSH:** DENA sends a message to a [service] in the [admin] whenever there's a change in a [person] in DENA
 - *a new [person] is enrolled DENA*
 - *a [person] deletes the [account]*
 - *a [person] changes some of the basic data (full name, contact data, etc)*
- **ADMIN PULL:** the [admin] retrieves the [person] data from DENA
 - On-line: DENA publishes a [REST service] where any identified [admin] can query [person] data
 - Individual [person data]: invoke the REST service using a [person id] (NIF) to get that [person]'s [data]
 - Multiple [person data]: invoke the REST service using a [person query] to get a **paginated [search result]** with the query-matching [person]'s [data]
 - Off-line (*batch for multiple*):
 - Pre-generated: DENA pre-generated files with [person] data
 - Bespoke: the [admin] requests a custom [person] data file that is generated by DENA for the [admin]

BEWARE that this [person sync] should not be confused with the [contact data] [data type]:

- The [person sync] is a means for the [admins] to have access to all the [person] account data in DENA
- The [contact data] [data type] is just another interoperable [data type] (similar to [admin procedure records] / *expedientes*) that allows a [person] to view their own [contact data] across multiple [admin] databases that store it.

This section details all the mechanisms that DENA offers the [admins] to have access to all DENA [person] data.

4.1 DENA Push

In the DENA-PUSH architecture, DENA just notifies the [admin] individually about any [change] about a [person] with a DENA account.

So, the only thing the [admin] must do is **provide a [person change push listener]** that receives that change notifications.

The change notification is a structure of type [DN00PersonSyncPushToAdminNotification] that contains:

- **syncData** (DN00PersonSyncData): information about why this [person] is pushed, in particular the **sync event** (DN00PersonChangeEvent)
- **person** (DN00Person): the full [person] data

```
{
  "syncData" : {
    "personRef" : {
      "id" : "11111111H",
      "oid" : "31289B53-0F4D-4A44-B96E-F6FD3A929E8D"
    },
    "personHashes" : {
      "nameHash" : "96d9632f363564cc3032521409cf22a852f2032eec099ed5967c0d000cec607a",
      "surname1Hash" : "799ef92a11af918e3fb741df42934f3b568ed2d93ac1df74f1b8d41a27932a6f",
      "surname2Hash" : "6627835f988e2c5e50533d491163072d3f4f41f5c8b04630150debb3722ca2dd",
      "fullNameHash" : "0d8ae2d653e8ea2edfe6ff5f50d0fd4090a3a51874434aefc61bf527b24369eb"
    },
    "createDate" : "2026-08-26T12:04:53.1869089Z",
    "lastUpdateDate" : "2026-08-26T12:04:53.1869089Z",
    "syncEvent" : "CREATED"
  },
  "person" : {
    "oid" : "31289B53-0F4D-4A44-B96E-F6FD3A929E8D",
    "id" : "11111111H",
    "lastChangeEvent" : "CREATED",
    "name" : "John",
    "surname1" : "Doe",
    "surname2" : "Smith",
    "contactInfo" : {
      "emailChannels" : [ {
        "addr" : "John.Doe@dena.com",
        "default" : false,
        "private" : false,
        "usage" : "PERSONAL"
      } ],
      "phoneChannels" : [ {
        "number" : "+34666555444",
        "availability" : "[12..15]",
        "default" : false,
        "private" : false,
        "type" : "MOBILE",
        "usage" : "PERSONAL"
      } ],
      "private" : false
    }
  }
}
```

4.2 Admin PULL

In the [admin-pull] architecture, the [admin] is proactively retrieving [person data] from DENA.

There are many options.

4.2.1 Online Admin pull

In the [online admin pull] architecture, the [admin] uses a DENA-provided REST service that can be used by an [admin] to get:

- Individual [person data]: invoke the REST service using a [person id] (NIF) to get that [person]'s [data]
- Multiple [person data]: invoke the REST service using a [person query] to get a **paginated [search result]** with the query-matching [person]'s [data]

4.2.1.1 Individual [person data] GET

- **PRE-CONDITION:** obtain a DENA [access token] (*see the [admin]-to-DENA auth section in the [architecture] document.*)
- **POST** to **api/admin/persons/{oid}** or **api/admin/persons/{id}** with an [interop message] containing a [payload] with just a [person reference], so if fetching using the [id], the [payload] is:

```
{
  "payload" : {
    "id" : "88888888Y"
  }
}
```

- The **response** to this post is directly a **JSON with all the [person] data**

```
{
  "oid" : "A9756ADB-3C66-4520-AEC1-6F0DC3D0F612",
  "id" : "00000000T",
  "lastChangeEvent" : "CREATED",
  "name" : "John",
  "surname1" : "Moshe",
  "surname2" : "Smith"
  "contactInfo" : {
    "emailChannels" : [ {
      "addr" : "john-moshe@trashmail.com",
      "default" : false,
      "private" : false,
      "usage" : "PERSONAL "
    } ],
    "phoneChannels" : [ {
      "number" : "+34666555444",
      "availability" : "[12..15]",
      "default" : false,
      "private" : false,
      "type" : "MOBILE",
      "usage" : "PERSONAL "
    } ],
    "private" : false
  },
}
```

4.2.1.2 Multiple [person data] SEARCH

- **PRE-CONDITION:** obtain a DENA [access token] (*see the [admin]-to-DENA auth section in the [architecture] document.*)
- **POST** to **api/admin/persons** with a [payload] of type [DN00PersonInteropSearchRequestPayload] that contains the [person filter] and the required [paging position]:

```
{
  "dateRange" : "Instant:[2026-08-25T11:00:01.798455500Z..)",
  "paginationRequest" : {
    "firstItemPosition" : 0,
    "requestedNumberOfItems" : 10
  }
}
```

- The **response** to this post is a [SearchResult] structure with the required page items containing all [person] data

```
{
  "filter" : {
    "dateRange" : "Instant:[2026-08-25T11:25:45.962145500Z..)"
  },
  "endPosition" : 3,
  "requestedNumberOfItems" : 10,
  "startPosition" : 0,
  "totalItemsCount" : 100,

  "pageItems" : [ {
    "oid" : "33593C45-6E35-48A8-BF0D-B73DB66A8BA9",
    "person" : {
      "oid" : "33593C45-6E35-48A8-BF0D-B73DB66A8BA9",
      "id" : "00000000T",
      "lastChangeEvent" : "CREATED",
      "name" : "John",
      "surname1" : "Doe",
      "surname2" : "Smith",
      "contactInfo" : {
        "emailChannels" : [ {
          "addr" : "John.Doe@dena.com",
          "default" : false,
          "private" : false,
          "usage" : "PERSONAL"
        } ],
        "phoneChannels" : [ {
          "number" : "+34666555444",
          "availability" : "[12..15]",
          "default" : false,
          "private" : false,
          "type" : "MOBILE",
          "usage" : "PERSONAL"
        } ],
        "private" : false
      }
    },
  },
  {
    "oid" : "BBA24773-21C7-42BE-A354-E46C6338389B",
    "person" : {
      "id" : "00000001R",
      "oid" : "BBA24773-21C7-42BE-A354-E46C6338389B",
      "lastChangeEvent" : "CREATED",
      "name" : "Jane",
      "surname1" : "Smith",
      "surname2" : "Johnson",
      "contactInfo" : {
        "emailChannels" : [ {
```

```

        "addr" : "Jane.Smith@dena.com",
        "default" : false,
        "private" : false,
        "usage" : "PERSONAL"
    } ],
    "phoneChannels" : [ {
        "number" : "+34666555444",
        "availability" : "[12..15]",
        "default" : false,
        "private" : false,
        "type" : "MOBILE",
        "usage" : "PERSONAL"
    } ],
    "private" : false
},
}
}, {
    "oid" : "4BCFB908-8B95-4893-98C2-960617374578",
    "person" : {
        "id" : "11111111H",
        "oid" : "4BCFB908-8B95-4893-98C2-960617374578",
        "lastChangeEvent" : "CREATED",
        "name" : "Alice",
        "numericId" : 0,
        "surname1" : "Baptist",
        "surname2" : "Williams",
        "contactInfo" : {
            "emailChannels" : [ {
                "addr" : "Alice.Baptist@dena.com",
                "default" : false,
                "private" : false,
                "usage" : "PERSONAL"
            } ],
            "phoneChannels" : [ {
                "number" : "+34666555333",
                "availability" : "[12..15]",
                "default" : false,
                "private" : false,
                "type" : "MOBILE",
                "usage" : "PERSONAL"
            } ],
            "private" : false
        },
    },
}
} ]
}

```

4.2.2 Offline

If an [admin] needs multiple [person data], the online approach using a query to get paginated search results is not always the best solution, especially if there's a lot of records to be pulled

As an alternative, DENA offers an offline (batch) mechanism that allows an [admin] to **pull a file with the records**.

There are two options:

4.2.2.1 DENA Pre-generated files

Pull a DENA-pre-generated file: DENA periodically pre-generates some files with [person data] for retrieval by the [admin]

- A file with all the [person] with a DENA account
- A file with all the [person] whose [data] has been changed in the last X time (e.g. *past day, past hour...*)

The [admin] simply calls the REST service requesting the specific pre-generated file, which is returned directly in the response payload.

The steps are simple:

- **PRE-CONDITION:** obtain a DENA [access token] (*see the [admin]-to-DENA auth section in the [architecture] document*).
- **POST** to **/pregens/asset** with a [payload] of type [DN00PersonInteropFetchPreGenExportAssetRequestPayload] that contains the pre-gen asset specification:

```
{
  "jobType" : "ALL_PERSONS",
  "exportType" : "DATA",
  "fileFormat" : "CSV",
  "hourOfDay" : "20"
}
```

The fields are:

- Job Type:
 - ALL_PERSONS: returns all the [persons]
 - UPDATED_PERSONS_SINCE_LAST_SUCCESSFUL_JOB: returns only person with changes since the last time the [pre-gen] file was generated
- Export type: the contents of the export file
 - DATA: all the [person] data
 - SYNC: no [person] data, just IDs & OIDs and timestamps
- File format: SQLITE / CSV / ZIP_OF_JSON / PARQUET
- HourOfDay: the [pre-gen] files are generated every hour, if the [admin] sends this field, it returns the file [pre-generated] at the hour if possible. If null, it returns the last available file
- The **response** to this post is directly a **byte stream** with the file

4.2.2.2 [admin]-requested bespoke files

Pull an [admin]-defined (*bespoke*) file: the [admin] requests DENA to generate a file containing [person data] that matches [admin]-defined criteria

The [admin] submits the **filter criteria**, and the REST service returns a [**job token**] because DENA have to generate the required file which takes some time.

The [admin] must **periodically poll DENA using this [job token]** to check if the file is ready for download

The steps are:

- **PRE-CONDITION:** obtain a DENA [access token] (*see the [admin]-to-DENA auth section in the [architecture] document*).
- **POST** to **/bespokes** with a [payload] of type [DN00PersonInteropCreatePullAdminBespokeRequestPayload] that have two versions:

1. If *requesting* ALL PERSON DATA:

```
{
  "orgAdminRef" : {
    "id" : "admin1"
  },
  "exportSpec" : {
    "personExportSpec" : "data",
    "lastUpdateRange" : "Instant:[2026-08-24T21:19:41.314878600Z..)",
    "exportContentSpec" : {
      "exportDefaultContactData" : true,
      "exportOtherContactData" : true,
      "exportFinData" : true
    },
    "exportFileFormat" : "CSV"
  }
}
```

The fields are:

- orgAdminRef: The orgAdmin requesting the [data]
- exportSpec:
 - personExportSpec: always “data” in this structure
 - lastUpdateRange: include [persons] that have changed in this date range
 - exportContentSpec: “blocks” of [person] data to be included in the file
- exportFileFormat: the file format (SQLite/CSV/ZIP_OF_JSON/PARQUET)

2. If *requesting* just SYNC DATA: IDs & OIDs and timestamps

```
{
  "orgAdminRef" : {
    "id" : "admin1"
  },
  "exportSpec" : {
    "personExportSpec" : "sync",
    "lastUpdateRange" : "Instant:[2026-08-24T21:19:41.482076Z..)",
    "syncEvent" : "CREATED",
    "exportFileFormat" : "CSV"
  }
}
```

The fields are:

- orgAdminRef: The orgAdmin requesting the [data]
- exportSpec:
 - personExportSpec: always “sync” in this structure
 - lastUpdateRange: include [persons] that have changed in this date range
 - syncEvent: the [person] change event (CREATED/DELETED/UPDATED/ID_CHANGED)
- exportFileFormat: the file format (SQLite/CSV/ZIP_OF_JSON/PARQUET)

- Since DENA takes some time to generate the requested file, the **response** to this POST is a [**job token**] that the [admin] can use to query DENA to know the [job] status. The [payload] of the **response** reflects the [status] of the [job]: **registered** and awaiting to be processed

```
{
  "oid" : "83F7BADC-1C54-4E80-8DB5-A9913ED3D41F",
  "admin" : {
    "id" : "admin1"
  },
  "exportSpec" : {
    "personExportSpec" : "data",
    "lastUpdateRange" : "Instant:[2026-08-24T22:02:06.508950400Z..)",
    "exportContentSpec" : {
      "exportDefaultContactData" : true,
      "exportOtheContactData" : true,
      "exportFinData" : true
    },
    "exportFileFormat" : "CSV"
  },
  "processingAttempts" : 0,
  "registeredAt" : "2026-08-25T22:01:06.5551369Z",
  "status" : "REGISTERED"
}
```

- Using the [job oid] ("**oid**" : "83F7BADC-1C54-4E80-8DB5-A9913ED3D41F") returned in the previous step, the [admin] can POST **/bespokes/{job0id}** to **get updates about the job processing [status]**
 - If the [job] is still **registered** but not yet started, the response is **the same** as the response of the [job creation]:

```
{
  "oid" : "83F7BADC-1C54-4E80-8DB5-A9913ED3D41F",
  "admin" : {
    "id" : "admin1"
  },
  "exportSpec" : {
    "personExportSpec" : "data",
    "lastUpdateRange" : "Instant:[2026-08-24T22:02:06.508950400Z..)",
    "exportContentSpec" : {
      "exportDefaultContactData" : true,
      "exportOtheContactData" : true,
      "exportFinData" : true
    },
    "exportFileFormat" : "CSV"
  },
  "processingAttempts" : 0,
  "registeredAt" : "2026-08-25T22:01:06.5551369Z",
  "status" : "REGISTERED"
}
```

- If the [job] processing has **started** but not yet finished

```
{
  "oid" : "83F7BADC-1C54-4E80-8DB5-A9913ED3D41F",
  "admin" : {
    "id" : "admin1"
  },
  "exportSpec" : {
    "personExportSpec" : "data",
    "lastUpdateRange" : "Instant:[2026-08-24T22:02:06.508950400Z..)",
    "exportContentSpec" : {
      "exportDefaultContactData" : true,
      "exportOtheContactData" : true,
      "exportFinData" : true
    },
    "exportFileFormat" : "CSV"
  },
  "processingAttempts" : 1,
  "registeredAt" : "2026-08-25T22:01:06.5551369Z",
  "startedAt" : "2026-08-25T22:01:46.7589852Z",
  "status" : "BEING_PROCESSED"
}
```

- When the [job] processing **finish**

```
{
  "oid" : "7F46C5E1-7774-4C56-A450-5AD788D33EB0",
  "admin" : {
    "id" : "admin1"
  },
  "exportSpec" : {
    "personExportSpec" : "data",
    "lastUpdateRange" : "Instant:[2026-08-24T22:04:25.844500800Z..)",
    "exportContentSpec" : {
      "exportDefaultContactData" : true,
      "exportOtheContactData" : true,
      "exportFinData" : true
    },
    "exportFileFormat" : "CSV"
  },
  "exportResultAssets" : [ {
    "exportContentSpec" : {
      "personExportSpec" : "data",
      "lastUpdateRange" : "Instant:[2026-08-24T22:04:26.072403500Z..)",
      "exportContentSpec" : {
        "exportDefaultContactData" : true,

```

```

        "export0theContactData" : true,
        "exportFinData" : true
    },
    "exportFileFormat" : "CSV"
},
"fileStoreItemOid" : "D824157A-CB93-4FC3-99ED-F567513FB1A2"
} ],

"processingAttempts" : 1,
"registeredAt" : "2026-08-25T22:03:25.8726521Z",
"startedAt" : "2026-08-25T22:04:06.0708960Z",
"finishedAt" : "2026-08-25T22:04:16.0708960Z",
"status" : "FINISHED_OK"
}

```

Note that the if the [job] processing has FAILED (status=FINISHED_NOK), the processing is **retried until a max number of attempts is reached**. If that circumstance arises, then the status=FINISHED_NOK_TOO_MANY_ATTEMPTS

- The [admin] should keep **polling /bespokes/{jobOid}** from time to time **until the [job] finished OK** in which case the [admin] must POST **/bespokes/{jobOid}/asset** to **download the generated file**

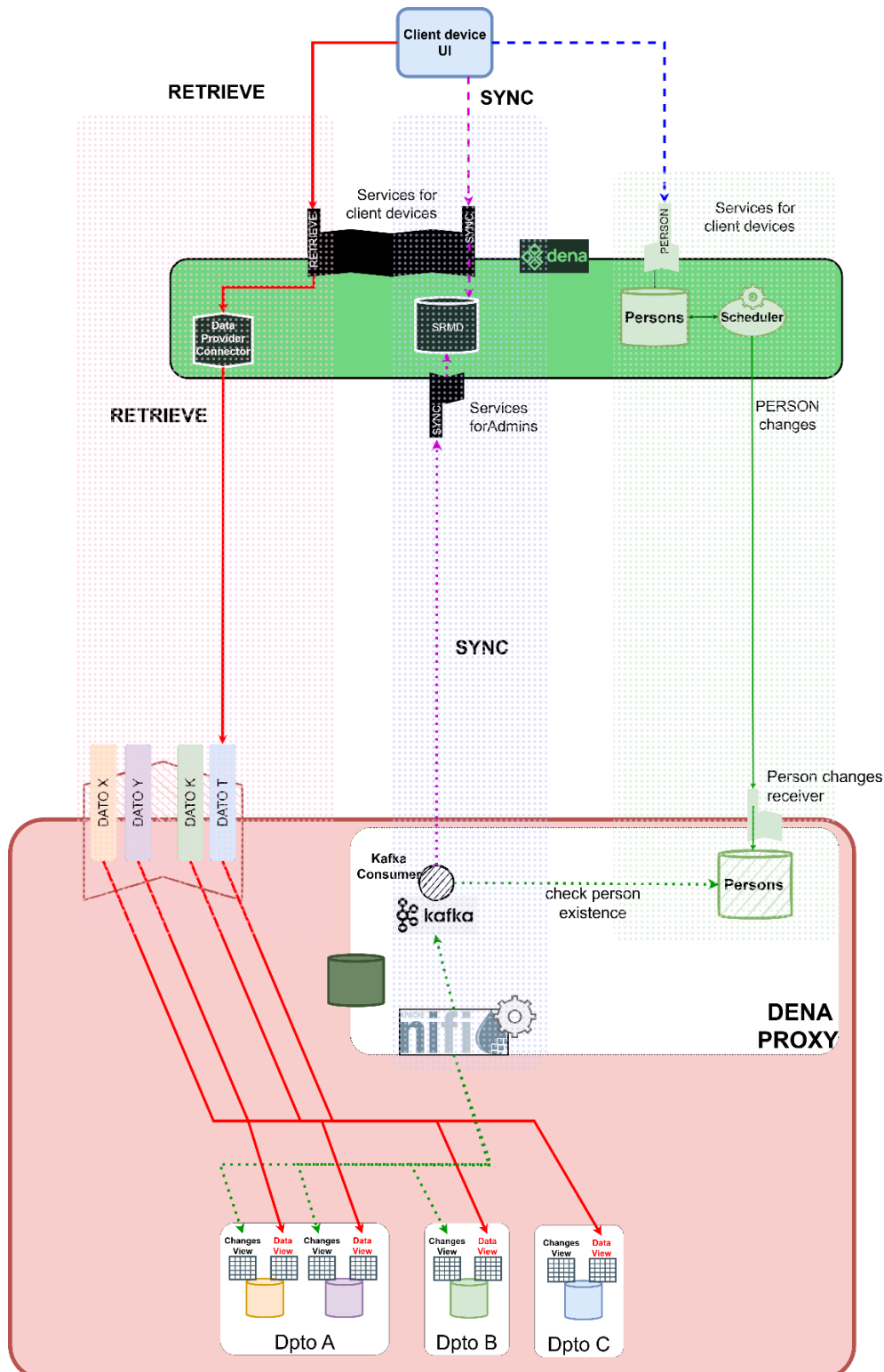
5 INTEGRATION REFERENCE ARQUITECTURES

Many [admins] have dozens of [data types] and [data origins] to integrate in DENA and a one-by-one approach is not the best solution because it leads to multiple different components doing the same thing (for example, multiple components sending SRMD sync meta-data to DENA)

So a more *centralized* approach can help the multiple [data origins] integration in DENA.

The following image depicts one approach where the three areas of integration in any [admin] are represented:

1. Person sync
2. SRMD sync metadata sending
3. Data retrieval [data provider] services



5.1 Person Sync

The [person sync] is usually done **once-per-admin** so every DENA-integrated [admin] has a **single replica** of the [person] DB

This integration is straight forward using the DENA provided artifacts that can be deployed in any [admin] datacenter

5.2 SRMD sync metadata sending

The first thing a [data origin] has to do to integrate in DENA is to **send changes**. To do so, the [admin] can deploy a **centralized changes collector** using [Apache NI-FI] that simply monitors a [DB view] of every [data origin] for changes and when detected sends a [message] to an [apache kafka] with the changes who just send the changes to DENA

5.3 Data Retrieve

For the [data retrieval] the [data origin] data (*or a copy of it*) has to be accessed directly

The easiest way is to ask the business to create a [DB view] of the [data] to be shown in DENA (*that usually is just a few columns of the whole business data*) and deploy. This [DB view] is the source of a [data provider] service.

Many times a convenient way to deploy ever [data provider] service is to group all the [data origin]'s [data providers] in single common service and distribute the data access using url paths:

<https://dena-internal.my-admin.eus/dena-data-provider/data-type1/byperson/{personId}>

<https://dena-internal.my-admin.eus/dena-data-provider/data-type2/byperson/{personId}>

<https://dena-internal.my-admin.eus/dena-data-provider/data-type3/byperson/{personId}>

...